



Transcript

Week 4: Data Analysis and Git Fundamentals

Video 1 - Data Analytics and Git Fundamentals

Are messy datasets slowing you down? Data inconsistencies, missing values, and outliers can impact analysis and decision-making. Let's explore effective techniques to identify and resolve these common data quality issues.

With the power of Pandas, you'll filter, sort, and aggregate data to reveal patterns and relationships hidden beneath the surface. Alongside this, you'll delve into the world of version control, learning how to set up and navigate a Git repository with essential commands like clone, add, commit, push, and pull—ultimately making it easier to manage changes and collaborate with others effectively.

Video 2: Introduction to Excel for Data Analysis

Excel serves as a user-friendly gateway to data analysis, thanks to its widespread availability and robust built-in features. It offers straightforward options for importing data, as well as tools for grouping, filtering, and pivoting—making it an ideal choice for those looking to perform basic to intermediate analyses.

The first step in data analysis is importing your data.

Excel makes this easy by allowing you to import data from multiple sources, such as CSV files, text files, and databases. Using the "Data" tab and the "Get External Data" feature, you can seamlessly connect to various data sources, including online databases.

Grouping data in Excel helps organise and summarise large datasets, making analysis easier and more efficient. You can group rows or columns to spot trends, compare data, and draw insights. The process is simple—select your data, use the 'Group' option in the Data menu, and instantly structure your information. For example, you can group sales data by month or categorise products for a clearer view.

Pivot Tables are a powerful tool in Excel that allow you to quickly summarise and analyse large datasets with just a few clicks. They help organise data, making it easier to explore trends and insights. You can create a Pivot Table effortlessly and customise it by filtering, sorting and aggregating data to fit your analysis needs.

Video 3: Introduction to SQL for Data Analysis

Struggling with large datasets? Excel is great, but SQL takes data analysis to the next level. Excel works well for basic data tasks, but when dealing with large datasets or complex queries, SQL is a game-changer.

SQL is a powerful tool for querying and analysing large datasets efficiently. Like Excel but on a much larger scale, SQL's aggregation functions allow users to extract meaningful insights with ease. As the standard language for interacting with databases, SQL is widely



used in data science to access, manipulate and analyse stored data, making it an essential skill for handling vast amounts of information effectively.

SQL aggregation functions help you summarise data from multiple rows into one result. Functions like SUM, Average, COUNT,, Maximum and Minimum enable quick calculations for totals, averages, counts, and extreme values in a dataset. These functions are essential

for analysing data and providing valuable insights, such as finding total sales or the average age of a group.

The 'GROUP BY' clause in SQL allows you to group data based on one or more columns and then apply aggregation functions like SUM, Average, or COUNT to each group. For example, you can group sales data by product category and calculate the total sales for each category. The syntax is SELECT column, AGGREGATE_FUNCTION column, FROM table GROUP BY column; which helps you gain deeper insights into your data by summarising it in meaningful ways.

Are you ready to take your data analysis to the next level with GROUP BY? Think about it!

Video 4: Introduction to Git

Managing data and code changes is key to successful collaboration and analysis.

SQL is crucial for analyzing data, but when working on code or in collaborative projects, managing changes is just as important. Git is a powerful tool for tracking changes in your code and ensuring smooth collaboration. Version control, through tools like Git, is an essential skill for developers, making it easier to manage code changes and collaborate effectively in any development environment.

Version control is a system that tracks changes to code or files, making it easier for multiple people to collaborate. It saves every version, preventing data loss and enabling rollbacks to previous states if needed. This is especially useful in teamwork, where project files evolve over time and frequent updates are made. By maintaining a history of modifications, version control ensures that changes are well-managed and easily reversible.

Git is a distributed version control system that enables developers to track changes and collaborate on software projects. Originally created by Linus Torvalds in 2005 for managing the Linux kernel, it is now widely used across various development fields. Git allows every developer to have the complete project history, making it easier to work collaboratively and manage changes efficiently.

To begin using Git, first, install it on your machine. Git is available for Windows, Mac, and Linux, ensuring compatibility across platforms. After installation, you can create a new Git repository in any folder by running the git init command. This command converts your folder into a version-controlled project, making it the starting point for tracking changes. It's important to understand the distinction between local repositories, stored on your machine, and remote repositories, hosted online, as they play different roles in version control.



Once your repository is set up, Git provides essential commands to manage your code. You can clone a remote repository, add files to be tracked, commit changes with a message, and push updates to a remote repository. To stay up to date, use pull to fetch and merge changes from the remote repository.

In Git, a commit captures a snapshot of your project at a specific moment. Branching lets you create separate development paths for new features or experiments without affecting the main project. Use `git branch` to create a new branch and `git checkout` to switch between them.

Git makes team collaboration easy. Use `git clone` to copy a remote repository and `git pull` to stay updated with the latest changes. When multiple people edit the same file, merge conflicts can occur, but Git provides tools to resolve them. To keep projects clean, follow best practices: commit frequently with clear messages, use branches for new features, and always sync with the team's code.

Mastering Git isn't just about commands—it's about teamwork, efficiency, and keeping your projects organised. Start using Git the right way and collaborate like a pro!

Video 5: Git Basics: Version Control with GitHub – Demo

Hello and welcome everyone. In this demo, we are going to talk about the Git fundamentals. We are all aware that Git is a tool which allows us to version control our code.

And by this time, we are all aware that Git is very critical to software development. Working as a team member, collaborating with others, ensuring your code is managed properly is what is basically covered via Git. In this demo, we are going to use GitHub, which is completely free of cost.

And we are going to learn how do we commit our code to Git, manage it over there. So, the very first thing is you can sign in into your GitHub account. You can create your account and on the homepage, you would see your listed repositories on the left-hand side.

If you have signed in for the first time, this would be blank. You can click on this green button which says new, which means to create a new repository. And I'm going to name some repository.

Let's say I name it as emeritus demo. And I can provide a brief description. I would say my first journey into Git.

I select on this checkbox to add a readme file, which would allow me to initialise my Git repository with a readme file. And I press on this button, create repository. The moment I do this, you will see that a repository is created with the name you had entered, the tag as public.

And then you have the readme file, which says first journey to Git. Now, I would basically go to my local editor. I prefer using Visual Studio Code, but it can be any editor.

Ensure that you have the Git installed within that editor. I open a new folder. I would basically go with one folder, which is currently blank.



You can create an empty folder and you can open that folder. Now, I can simply create a file within this folder, say new file, a Python file. And I would simply say print hello world and save this file.

Let me give this a name. I would say test.py. Now, I would basically want to version control this file. So, I click on this terminal, open this terminal, and you'll see that I would locate with the same directory where my test.py file is located.

And I would start with some operation. I will say git status. The moment I say, it would initially say that it's not a Git repository, which means that I have to first initialise it and say git init, which is initialising an empty Git repository in the same location.

And now when I do git status, it will now explain me that on this branch, which is a master branch, there has no commits being made yet. And there is one file which is untracked. Now, I would want to add this file.

So, I'll say git add and the name of the file test.py. And again, if I go ahead with git status, you would see that now this file is being tracked. So, the colour turns green and this file is continuously tracked. Any changes you make to this file will be tracked by Git.

Now, because I have made the first commit, I would want to basically complete that. I would say git commit and I'll add my first commit as a comment over there. And it will explain you that it is a change made in one of these files and one insertion, which is correct because we have actually written this line.

Now, this is my local repository, but I have a Git repository already created. And I want to push my code over here in this Git repository. And this is where I will go back to my terminal.

And in fact, I would select this green button code. And this is the location of my report repository, which I will copy. I'll go to Visual Studio Code.

I'll say git remote add voltage and paste that URL over here. And that should have added my

repository. And when I check that, it says, yes, this is the repository which is added.

All what I have to do next is I just have to push my code. The moment I do that, you will see that the moment I do that, what you will see that the code is pushed. And if I go back here, you'll see that it says master had recent pushes three seconds ago.

I can click on this dropdown. Let me refresh that. I can click on this dropdown and I would see a master branch.

And it would actually contain the test.py code. You can open this and you can view your code, which is what you created locally. It's now on your remote repository.

Now, let's say that someone who is using this repository makes changes to this code. I would make a change. I would say my first commit on git.

And I would commit these changes on remote repository. But you will see that my local is the same file as it is. Now, this is where the management of our collaboration comes in picture.



Any change which is made in the local repository, how do we update our local repository with the change made in the remote repository? To do so, we can simply say `git pull origin master`. The moment you do so, you will see that this will automatically change to the most updated code on your repository. Now, if you further go and make any change, I would say `second commit over here`, save it.

Again, you see the status. You will see it says that there is one file which is modified. Add that file.

And again, commit that file with a message. I would say `second commit`. And push that.

The moment you do so, you will go back over here, click on this, and if you refresh that, you should be able to see the code again updated over here in the remote repository. And if you would want to see the history of the changes made to your code, you see it says the first commit, the updated desktop by second commit. And if you see, it also tells you that who updated.

This is something which was updated from my local. This was updated from GitHub. And this

was updated again from local.

And that is what happened. You can click on this. And you can view the code at that moment in time.

So, it tells you commit by commit. See the previous commits for that. So, this was the code at this moment in time.

And if you go to other, this would have been the code at the first instance. So, that is what we have in our repository. It talks about all the commits being made to your branch.

So, we have three commits. You can click on these three commits and you will get to see all

of these over here. And you can click on any one of them.

And that should tell you that what was the change made. So, this was the original and this was the updated. So, it tells you the green highlighted one, this was added.

Anything which is removed and anything which is added would be basically explained over here compared to the previous commit. That was cool. And I hope you use this tool to version your code.

It makes life a lot easier. Thank you everyone.

Video 6: Excel Basics: Data Analysis with Pivot Tables – Demo

Hello and welcome everyone. In this demo, we are going to learn how can we perform basic analysis using Excel. So, the first thing what we would want to do is to load the data into this Excel sheet for our analysis.

Assuming that I have a CSV or an Excel file saved somewhere in my local, I would want to locate that directory. So, I am just going to open and let us say this is the directory where I have my data located and this is one such data which is located. I am just going to open that and your Excel sheet should get populated in this looks more like an employee information data.

Now, we want to do some level of analysis being on Excel itself. So, the first thing what we do is we try to select the data on which we want to do any sort of analysis and let us start with the very basic analysis which you have seen earlier in your module in the chapter which we have gone through. We select on insert and you can see the first selection here which says pivot table.

I am going to click on this pivot table. You can see that the range which we have selected is pre-populated over here in case if you want to modify that range, you can modify it if you are aware of the Excel formatting. The next selection is basically trying to explain you where do you want to do this pivot analysis.

You want to use a completely new worksheet or an existing worksheet. I would prefer doing it on a new worksheet and I would click ok. The moment I do that a new worksheet opens up parallel to my data sheet and you also have some options on the right-hand side to perform operations.

Let us say I want to do a department wise analysis. I am going to drag this department in the rows column. The moment I do so, you see that we have the departments grouped and they are kept in rows.

Now, what kind of analysis I would want to do? Let us say I would want to perform analysis on salaries. I would want to know the salaries department wise. So, I will drag this salary and place that in values.

The moment I do so, by default, it will calculate the sum of salaries per department. In a way, it is telling us that finance department the total salary is this amount, HR department the total salary is this amount and so on and so forth. Not only this, you could also change this default behaviour of replacing this sum of salaries to some other operation.

You can select the count and you would see that this column is now changed and you have the count of instances per department. I would basically roll back to the sum itself and let us try to see if I can add more level of analysis in the adjacent column. So, again, as I said, I will go back and I will drag this salary further over here and I will select this drop down, click on value field settings and let us say I select the max and I can keep on adding adjacent columns over here.

So, again, this helps me to understand department wise what are the total salaries dispersed and the maximum salary dispersed. We can also create a chart, you can select this, you can go to this insert tab, you can see this pivot chart, click on this and you have



different types of charts available over here. I would basically go ahead with the default column or bar chart and I just need to click okay.

The moment I do so, you see a good-looking chart explaining these numbers in a visualised manner also. And you can do a department level and select any one of these, any two department and that should basically do the job. So, this is how we tend to perform analysis using group by operations, pivot tables and visualise them on Excel.

There are many other operations possible and I would leave that up to you to explore. That is all for this video. Thank you everyone.

Video 7: SQL Demo

Hello and welcome everyone. We are going to do a demo of basic SQL operations and we are going to have some sample tables. You can see on the right-hand side, we have the available tables as customers, orders and shipping tables.

The schema of these tables are defined on the left-hand side. The customer table has a customer id which is of an integer type data. Then we have the first name which contains a varchar type data.

Then we have the last name, another varchar type column. Then we have the age which is obviously an integer and the country which is going to hold a varchar type of data. Similarly, we have other two tables orders with order id, item, amount and customer id and shippings with shipping id, status and customer.

And you can see on the right-hand side the sample data which resides within the tables. You can also create more tables. You can also try to define the schema of those tables while you create them.

But for the sake of demonstration today, I would assume that we are already having these tables and we are going to see some SQL in action and we are going to use this online editor which would be very similar to any editor you might use. Let us start with the first query.

Assume that I would want to select all records from customers.

So, I can actually use the select query. And since I want to use all the records, I would say star from customers table. And I can run this SQL and the output should be all the records from the customer table.

So, you see this entire table which resides in my database is fetched and shown to me in the output. Now, what if I want to only select specific columns? For example, I want to select the first name and the country. So, I can modify this.

I can say first name comma country. And I run this SQL query and you would see that in the output you have the two columns, the first name column and the country column. And the same action you can also do on the other two tables.

Let me just quickly do a demonstration. Let us say I want to see the item and its amount, but from a different table that is orders. And you would see that I have the corresponding column item and the column amount.



Well, that is how you can select either the entire table or a specific column within the table. Now, I want to move ahead and I want to implement the where clause in SQL. So, let us try to see what this where clause does to our query.

Let us select this star from customers. I am going to select all the data for the customer table. Now, I want to filter this table based on a certain condition.

Let us say the condition which I want to use is I want to select only customers from UK. So, I can actually use the where clause where country equals to UK. And you would see when I execute that only those rows which contain country as UK are selected.

Similarly, if I want to do a similar sort of filtering on the orders table. For example, select orders table, but I want to say that where amount is greater than certain number. For example, I would select amount greater than 350.

So, I can say where amount and I would choose the greater than operator 350. And you would see that it would actually list me all those rows where the amount is greater than 350. Now, you want to select a specific column.

For example, you just want to select an item. I can mention that and I can run that. So, it will list me all the items for which the amount is greater than 350 from the orders table.

Further, there is another special function in SQL. Let us try to see that. I will again roll back to

my original query.

Let us say I have this query. Now, I want to order the list of rows by a certain column. For example, I want to order that by age.

So, I have select star from customers and I can say order by the age column. And I want to do that in a descending order. So, I can mention that.

So, if I run that, so you would see that it is ordered in the descending order. Now, assume that after ordering it, I want to just fetch the top three rows. So, I can add one more keyword over here limit and mention the number of rows you want to fetch in that order.

So, I am going to see the top three rows. So, we could order it by in a certain sequence and also limit it by a certain number. Further, there are some aggregate functions which you can apply in SQL.

So, I will roll back to this customers table. Or let me try to do that on orders table which would make more sense to us. Let us say these are all my orders.

Assume that I want to know the total number of orders. So, I can actually use the aggregate function count. And if I run that, it should give me the total number of rows or the total number of orders.

I can also add an alias over here. I can say as total orders. And you would see that it would actually change that name count star to total orders.

This is called as aliasing. Not only count, you can also use other aggregate functions. So, I will just roll back to orders.



And let us say I have this amount and I want to calculate the average order amount across the table. So, I can first select amount. So, I get all the amounts and then I apply the aggregate function average that should give me the average amount across the table.

And I can add an alias for my understanding. And it should give me the average amount as an alias and the actual amount calculated. Further, let us say I want to do some analysis.

You see that I have this table and I want to know that how much total did the customer ID 4 spent. So, I have 2 rows for customer ID 4 and I want to total the amount for customer 4. So, first of all, let me select customer 4. So, I will say where customer ID equals to 4, run that.

So, that gives me the 2 rows.

Now, I want to sum this amount. So, I can simply say sum amount and run this should give me the total sum. And you can also give it an alias something like that and that should give you the name and the amount.

Now, what if I do not want to do for a specific customer? I want to do it for all the customers. So, I just roll this back. Let us say this is my table.

Now, I want to group all my customer IDs and then know their total spend. So, what would I do is, I would use something called as a group by. So, for example, I would say customer ID give me the customer ID and the sum of amount spend by different customers.

I need to group them by customer ID. So, what I am doing here is, I am grouping by customer

ID. So, they would become one group and I would sum their amount.

This would remain as a single entity, this also remains as a single entity, this also remains as a single entity. So, when I run this, you would see 1, 2, 3, 4 because they were all single entities. So, they are mentioned as it is, but because we had 2 amounts for customer 4, they are now added because I had applied a sum as an aggregate operation.

Further, let us say I want to perform something called as inner join. Now, you know that I have 2 tables. This is my first table, orders and I have customers.

If you see these 2 tables, what I am interested in is, I am interested in combining these 2 tables. I am interested in understanding that the person John had purchased what item of what amount. To do so, I have this customer ID as a key which is a foreign key which connects these 2 tables.

So, I am going to use this customer ID to connect these 2 tables and I am going to show you that for each customer, what are the items and their respective amount that is what we call as join. So, to do so, first of all, let us try to, so I am going to see from the customers table, I am going to select the first name. From the order table, I am going to select item and amount.

Now, because I am aliasing, orders, so you see o.item actually means from the orders table get me the item, from the orders table get me the amount. So, I need to mention here



orders o for the compiler to know that I am referring orders as o and I am also referring customers as c. So, you see c.firstname and that is done. Now, I want to also explain that on what key should I combine them.

So, I am going to say customer ID on the order side and customer ID on the customers table. So, again if you see, what have I done here is I have, let me show that. So, what did I do? I have selected couple of columns from the order table and a single column from the customer table.

So, because you see this c dot, so I have to mention customer c because it should pick customers first name from the customer table and item and amount to be picked up from the order table. So, I alias that by o and I will use that as a reference over here. Now, I need to tell that for me to combine these tables which is that key which I should use.

So, that is what I am saying on the customer ID use the o.customer underscore ID and equate that with c dot customer underscore ID and when I run that. So, that so you see for each name I have the corresponding item and the amount. So, that is what we call as join.

You can also join in different manners. So, you would see here that this customer is not actually there in the table, Betty is missed. Why? Because in the orders table I do not have the customer ID 5. So, I do not know about Betty what item and what amount did she purchase.

Just in case if you want to list that if you want to list all the customers of the customers table then you would do a right join. So, you would say I would say right join customer c. So, the moment I do this right ok. So, looks like it is not supported in this SQL editor, but if you try that on your own editor you would see that it would list all the entries of this table.

Thank you.

Video 8: Advanced Reporting with Pandas

Hello everyone, welcome to this session. We're going to go through advanced reporting with Pandas using GroupBy and PivotTable. You have gone through what is GroupBy and PivotTable, helps you to aggregate and summarise your data.

We'll explore more options in GroupBy and PivotTable, how it could be useful. And also we'll work in terms of understanding what kind of output we are getting when we are using either of these aggregation options that we have in our hand. For this particular example, the dataset that we are using is called CustomerShoppingData.

It has invoice number, which is a unique ID for each transaction, customer ID, gender of the customer, age, category, which product category has been purchased, quantity, price, payment method, whether cash, credit card or debit card, invoice date, date of transaction, which is in the format of DDMMYYYY, separated by a slash, and the shopping mall name, name of the shopping mall, which is given over here. First, what we are doing, we are loading the data, importing Pandas as PD, and then providing our CSV file. The



invoice date column converted into a date time field so that we can extract some information out of it.

The format has been provided over here as %DMNY explicitly, so that Python understands first is the day, then the month, and the year four digit. We have also calculated what's the total price paid by the customer, which is nothing but multiply the quantity column with the price column. So at the time of loading the data, this much of work we have already done, and this is how the data look like.

We have invoice number, customer ID, gender, age, category, quantity, then price, payment method, invoice date, shopping mall, and then you have total price. Now, let's start our work with the group by operation first and understand what it does. I can say over here that let's take this data and find out what is the total spent by each gender.

So we can say `df.groupby`, I can say group it by given gender, and then pick up the column, which is called total price, which we have calculated, and sum that up. This is a very basic operation we do. So we got the sum for what's the male sum and female sum for the total price.

Understand one thing here, the result that you are getting over here is a series. Group by when you run, the output will always be a series. So this is a series with indexes as male and female.

That's the index. And the values are the sum of total price. That's what we get over here.

Another example, we can say `df.groupby`, I can say group it by category, pick up total price, and find the sum. So for each category, what's the total sum? That's what it gives us. So we can use group by in this manner that we have done earlier.

Interestingly, we can also group it by multiple categories, multiple variables. For example, find sum of quantity by gender and category. That's what I want to do.

So I can say here `df.groupby`, and this time I want to group it by more than one variable. So I'll provide a list of variables. I'll say category, and then within category, do a further split in gender.

Pick up the quantity column. Let's say I want to find the quantity sum and sum it up. So this is what I'm doing here.

So category, gender, quantity, probably see here quantity. Okay, and hold on a sec. Let's see.

This is a mistake here. The bracket will be closed here, and this will be outside. So that was a mistake here.

So now we got for each category for different gender, what is the sum of the quantity? And when I look at this result, I start feeling that this looks like a data frame, but it is not. It is still a series. Remember that.

But now we have multiple indexes, multiple indexes. Let me tell you how. I take the same code, and I'm going to store in a new variable here.

So I'm going to say here result is equal to this. And now I run type of result to know what kind of object we have created. It's a series.

So although it looks like a data frame, probably some of you might feel it looks like a data frame. I have this column, then this column, and then probably this column. But it's still series.

And interestingly, if you look at the index for this object, you will say it's multi index. Combining these two together, you are getting a unique index. And that's what is happening here.

This is the result we got. Now, what if I want to change the format of it? Or sometimes I want to change it because I want to do some more analysis on top of it. Or I want to show this table to somebody.

So what we can do, reset the index. So I can say here result, which looks like this right now. And I can say dot reset underscore index.

And once I do that, what it does is the existing indexes will be added as columns. Now you'll get a data frame. And the indexes will be added as a column.



So now you get category as a column, gender as a column, and then some of the quantity over here. And new index will be created. So when I reset the index, existing index will be added as a column.

If I run it one more time, 0, 1, 2 will be added. So resetting the index helps you to convert this result into a data frame, which I can store it as result underscore df is equal to this. Or probably result equal to this.

So that I can overwrite the original result object. So this way what happens, now I get a data frame, which I can further use for some purpose if required. See here.

That's a benefit of it. So multi-indexes you get. Understand, every time you're using GroupBy, it will produce a series.

It will never produce a data frame. And that's where the main difference between GroupBy and PivotTable is. PivotTable will create data frames.

Now we can also use multiple aggregate functions. How do we do that? We can say `df.GroupBy`, and I can say here group it by category. Pick up the quantity column.

And `.agg`, as in aggregate. And here provide a list of functions you want to use. I can say here start with men.

Then I want mean. I want max. I want standard deviation.

And when you run, it creates this particular result. Now this result will be a data frame because you have multiple aggregation. So I have, these are the index for the result for the data frame.

And these are the columns of the data frame. So we have one, two, three, four columns. Four columns are there, and we have the index as the category name.

That's a data frame. So I can have multiple aggregation also that you can do with GroupBy. Multiple aggregation we can do.

Or interestingly, we can apply some named aggregation also. We can say named aggregation. Let's try that.



Let me tell you what it does. I can say here `df.GroupBy`, and I can say group it by category. And then I say `.agg` directly.

I'm not writing any column name. I'm simply writing `agg`. And I can say here that I need an aggregation which is, say, total price.

Or I can say revenue, probably. Revenue is equal to, and here you provide a tuple, where which column you want to work with. I want to work with total underscore price.

That's a column name. And then second option is what function you want to apply, which is `sum`. So now I'm able to provide a custom name here.

Revenue is equal to a tuple. And the first element of the tuple would be the column name in your data frame. And the second would be which function you want to apply on that.

And then you can have multiple such aggregations. I can say `average underscore quantity`, which is equal to another tuple, use quantity column, and the function would be `mean`. Now it's a data frame with categories as indexes, sum of the revenue, total revenue, and average quantity.

So I can run multiple aggregation, and I can apply different functions on different columns. Or even if you want, you can also use the same column twice as, I can say here, another one would be total quantity. And one more time I will use this quantity column, but this time I will use the function as `sum`.

So that's another thing you can do with it. You can group by by a particular category, like in this example, incidentally, the name of the column itself is category. You can aggregate different columns by different functions.

That's what you can do over here, which makes it even more useful in our real-life examples. And this is how I'm able to aggregate our data. Now what I can do here is I can say that this entire data is my report, not data, but the group by that we have done.

And once I have created the report, I can say `report.to underscore CSV`. So I'm exporting it. So whatever aggregation we have done, if I want, I can export it, export the report.



We can create some charts also on top of it, and we can export the chart as well. Something export to CSV and the name I'll give as report.csv. And I'll run this, which will store this particular result in a CSV file on my system. Or I can write the entire Python code that connects to my email server and send this report to so-and-so.

And then I can schedule this to run every day in the morning, where it will read the data from some database or some sources, create the report, it will send automatically. At the moment, I'm simply exporting the data that we have aggregated. And over here you will see report.csv has been created.

See this? And I can use Excel file or I can use some other formats as well. So this comes very handy when you want to use group by and you want to do different aggregation with custom name. I've also shown you I can group it by multiple categories also, categorical columns.

Now let's see what PivotTable can do for us. PivotTable is very much doing the same thing, but how it's different. First of all, by default, any kind of aggregation you do with the PivotTable, it will create a data frame.

Second, it is a little more flexible in terms of the layout that you're looking for. So I can say here `pd.PivotTable` and data is equal to `df` index means what you want by row or what you want vertically distributed. So I'm saying category, that's a column in my data.

And then I say here values, which column you want to aggregate. I'm saying I want to aggregate quantity. And then it asks what's the aggregate function you want to use.

I want to use sum. And when I run this, it produces exact same result in terms of the values, what group I would have done. But the difference is it's a data frame.

For the same operation, if I do with the group byte, it will give me a series, but here it's a data frame. It has a single column called quantity, and these are my indexes. But what if I want my indexes to come as a column? Again, I can do a reset index.

Reset underscore index if required. Added as a column now, category and total quantity. So now pivot table, this part is very similar to what a group byte does, then how it's different.

This can help you with different layouts. Like I can say here, I can take the same example, and I can see here that index is category and add, let's say, one more argument here,



columns is equal to, and I say gender. And now when I run this, it gives me this kind of layout.

For each category, what's the sum of quantity for female and male? And this is not something you can get from group by directly. You'll have to reshape it, or you have to melt it. But here you get it easily with this option.

I'm saying category, what's the female sum, what's the male sum? That's what we can get over here. And just like group by, you can also use multiple aggregations. You can provide the list of aggregations over here.

And if I'm not resetting the index, let's look at it. What happens with this if I'm not resetting the index? It comes like this. Category would be your index, and columns will be male, female, and what's the sum? So then the category would be index over here, not a column.

So PivotTable makes it easier for you in terms of achieving different layouts. That's what the main benefit is. So when you want to cross tabulate and do some aggregation, it comes very, very handy.

The only problem with the PivotTable is that you cannot provide custom names over here. The custom names part, you have the ability in group by, but not in PivotTable. The custom column names is something which is very handy in group by.

But then here I can get different layouts. Like another report I can generate here, if you look at our data, we had shopping mall as well. So now we can do something like this.

We can say here PivotTable index. I'll provide a list of values. I'll say here we have shopping underscore mall as the first index value, category as second, rest I'll leave as it is.

So now I get something like this. For each shopping mall, for different categories, what's a female sum and what's a male sum? That's what I'm getting over here. So this is the benefit of your PivotTable.

So in different scenario, different kind of layout you can use. And if I export as an Excel file, it will come exactly like this, like a PivotTable in Excel. So depending on the situation, you can choose either of the aggregation method.



Most of the job that PivotTable does can be done with the group byte, but some of them are very unique to PivotTable. Other way around, anything that group byte does, PivotTable can do it. Apart from the custom names that you provide in group byte.

Here what happens if you want to provide custom names, like here female and male is coming, you have to store it as, I can say, total quantity or I can say quantity underscore report. And once you run this quantity report, you will realise that it has only two columns, quantity.report.columns underscore report, actually. It has only two columns, male, female, male.

And that I can change it to, I can say quantity underscore female and quantity underscore male. I have to manually do it. And then only it can change.

Let's see here. So that's something you have to do it after you have done the changes or created the report. So if I have to summarise between group byte and PivotTable, I would say both of them support multiple aggregations.

You have multi-index columns in both the ones, group byte also PivotTable. But when it comes to custom column names, group byte directly do it. But PivotTable, you have to do it afterwards, after creating the report.

And the use cases, group byte, programmatic group summarises and PivotTable creates Excel style reporting tables. And remember, all these reports that we are generating, wherever we want, we can export it in either Excel, CSV, JSON, or any other format. So that's your going into detail of PivotTables and group byte.